

東南大學
SOUTHEAST UNIVERSITY

Agentix: An Efficient Serving Engine for LLM Agents as General Programs

Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez and Ion Stoica
University of California, Berkeley

USENIX Symposium on Networked Systems Design and Implementation (NSDI'26)

汇报人：张靖周

2026 年 10 月 19 日

作者团队



Michael Luo

- UC Berkeley EECS 博士
- 导师 Ion Stoica

研究方向：

- AI Serving System
- LLM Post Training

UC Berkeley Sky Computing Lab

聚焦云计算，分布式系统和 AI Infra

vLLM (SOSP'23)

SGL (NeurIPS'24)

- (NSDI'26) Agentix: An Efficient Serving Engine for LLM Agents as General Programs
- rllm: an open-source framework for training language agents with reinforcement learning
- (ATC'24 Best Paper) Starburst: A Cost-aware Scheduler for Hybrid Cloud

提纲

1 研究背景与现状

2 研究机遇与挑战

3 研究内容

4 实验与总结

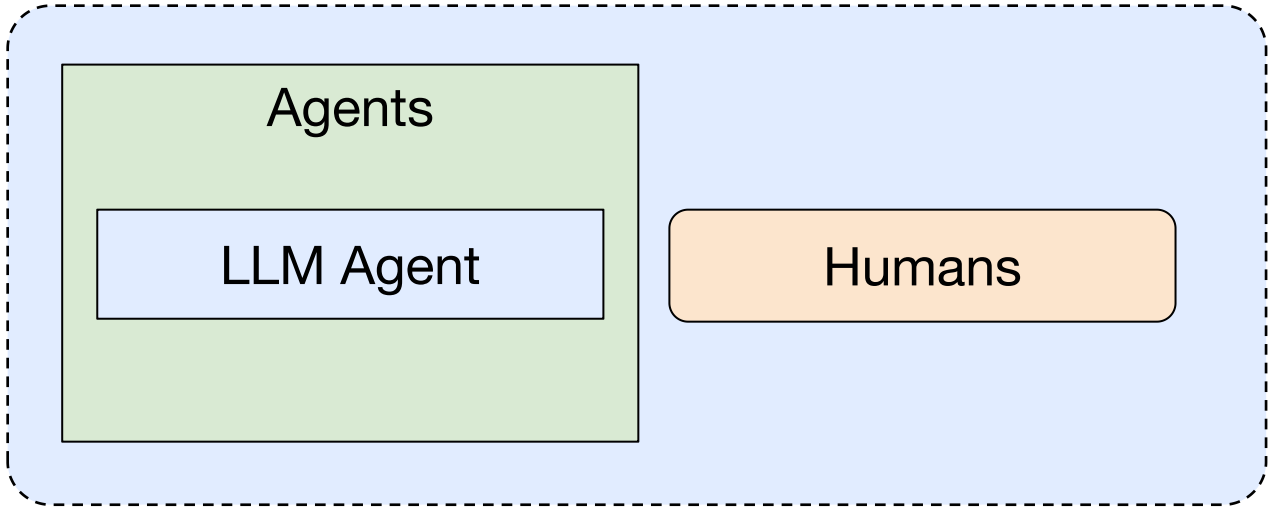
提纲

1 研究背景与现状
2 研究机遇与挑战
3 研究内容
4 实验与总结

背景：AI Agent Infrastructure

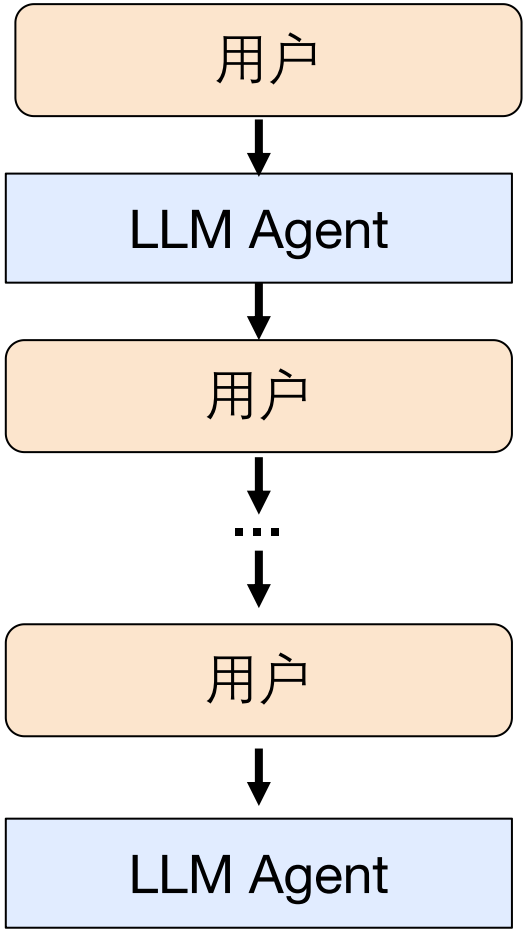
单线程

Chatbot



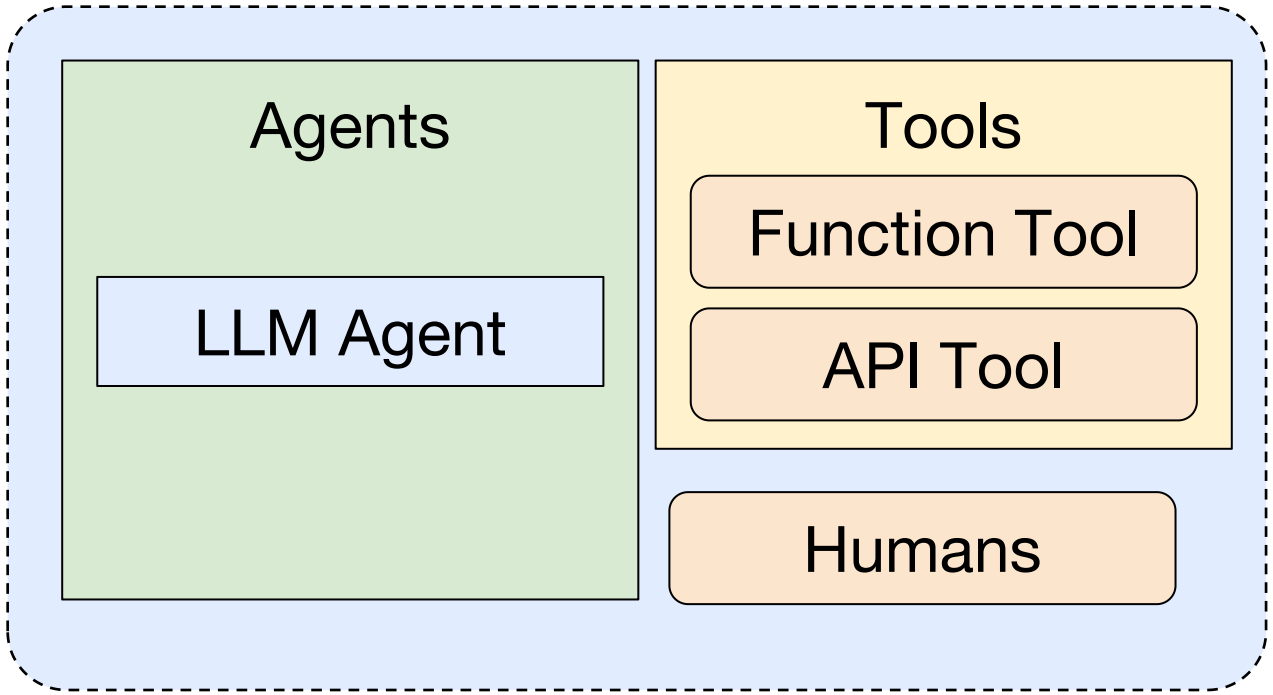




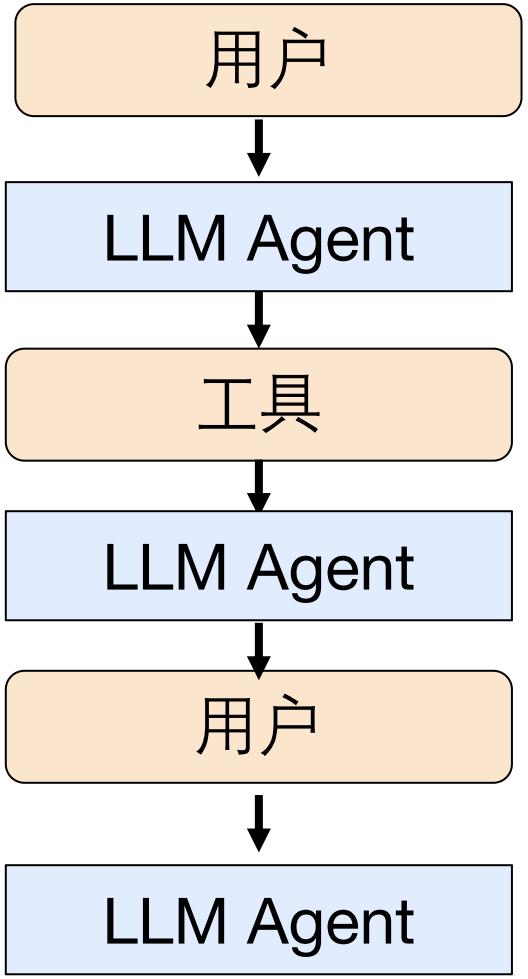


背景：AI Agent Infrastructure

单线程

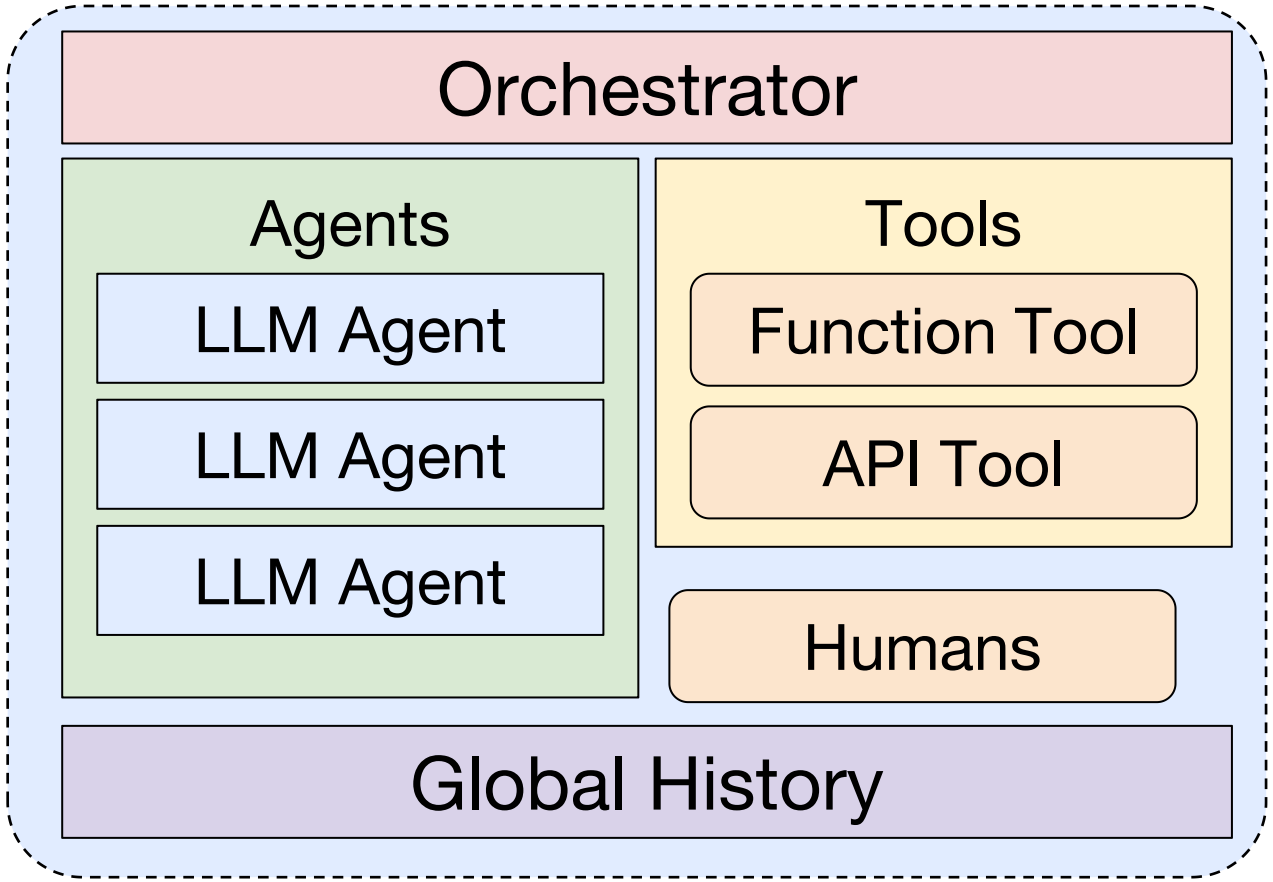


ReAct Agent

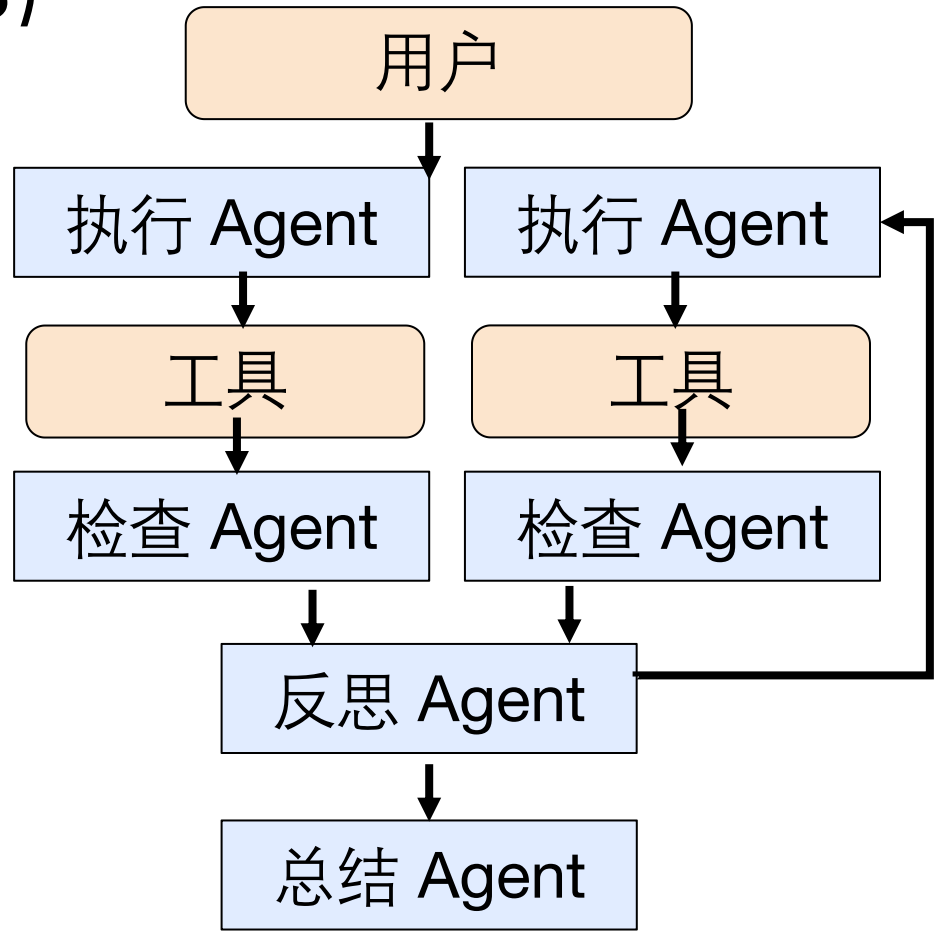


背景：AI Agent Infrastructure

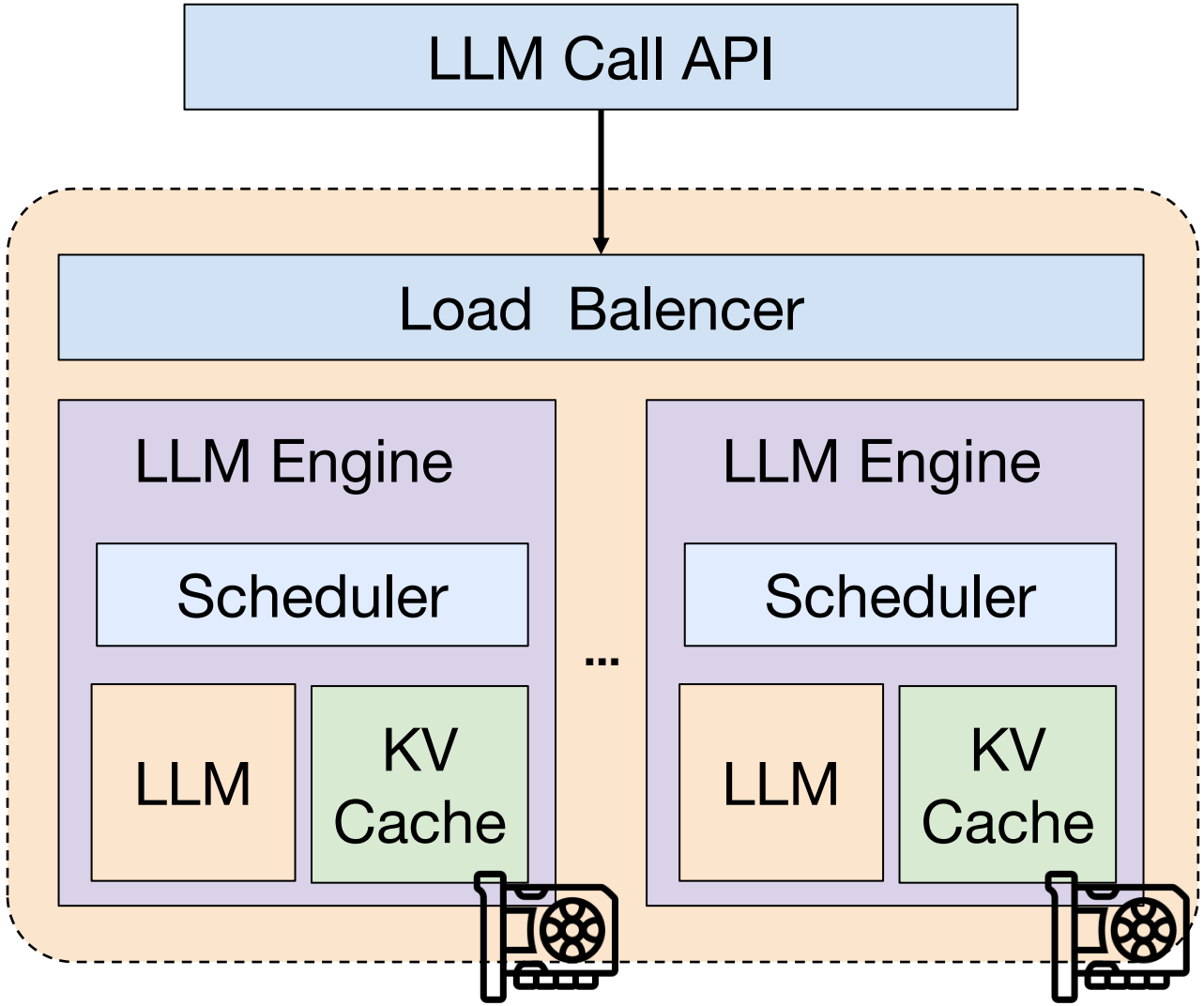
多线程



Multi-Agent（MCTS）

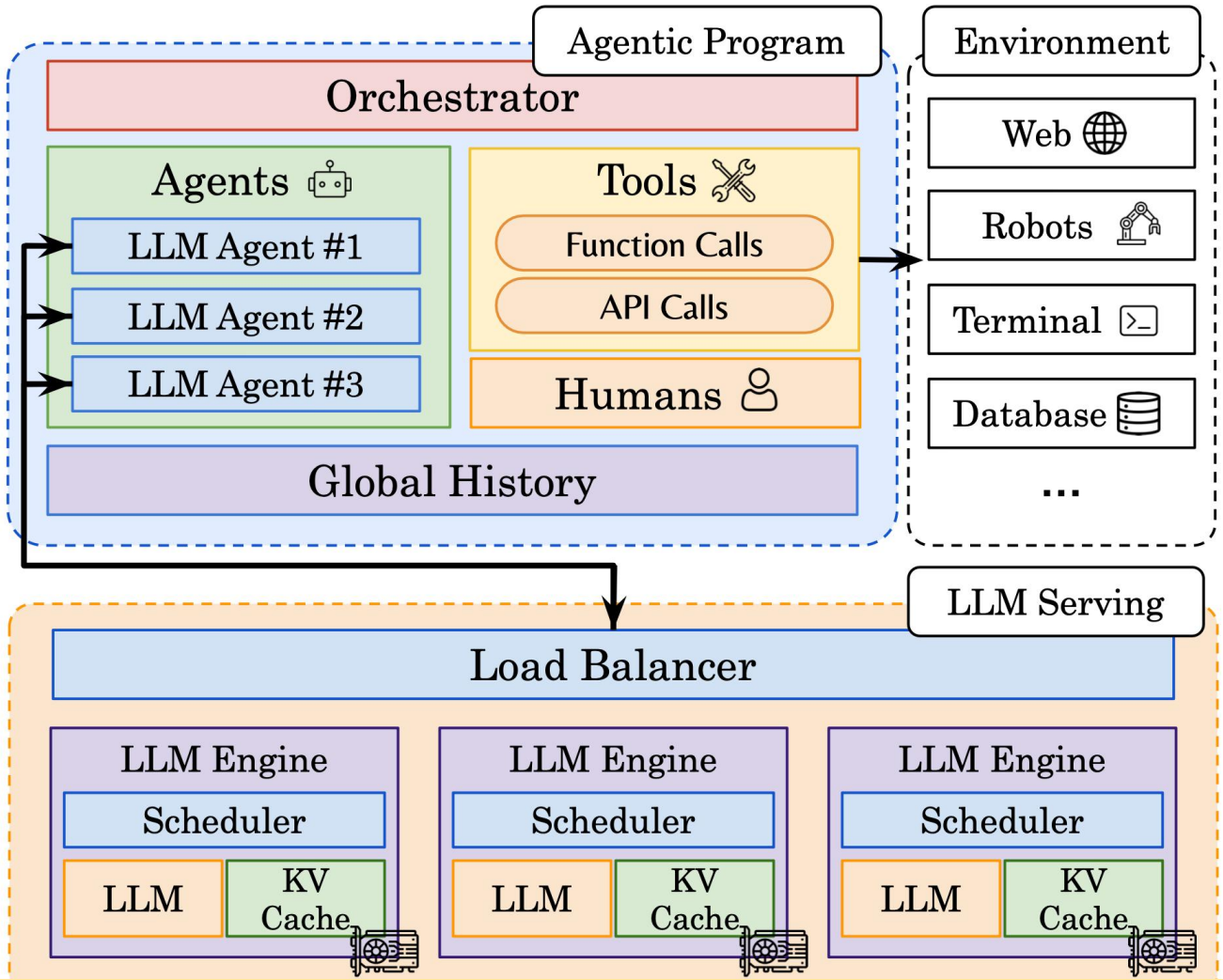


背景：LLM Serving



- **LLM Call** 一次 LLM 请求
- **Load Balencer** :多 GPU 负载均衡
- **LLM Engine** 一个 GPU 上部署的推理引擎
- **Scheduler** ： 调度 GPU 上需要推理的 LLM Call

背景



- 应用层多样化发展
- 将一个包括多个 LLM Call的工作流称为一个 Agentic Program

- 在系统推理层的是模型调用（LLM Call）级的支持

Problem：如何对执行结构多样且动态变化的 **LLM Agent** 应用进行推理调度优化，以降低端到端延迟？

相关工作

技术上

场景上	LLM 推理调度服务	调用级优化	程序级优化
	独立请求或 固定特定工作流	Web服务器、数据库等的 静态配置策略	贝叶斯优化、多臂老虎机 算法、因果推断
	Agent 动态场景	手工静态配置、资源超配 （默认Linux 完全公平调 度器；静态的进程资源分 配）	**Agentix (本研究)**

L1：现有的LLM 推理调度优化没有针对Agent 应用的**动态结构多样性进行优化**，通常假设Agent程序内的调用无关或者或者具有固定结构

L2：现有优化的方法都是从调度级别考虑，而**没有充分利用到程序级的信息**

提纲

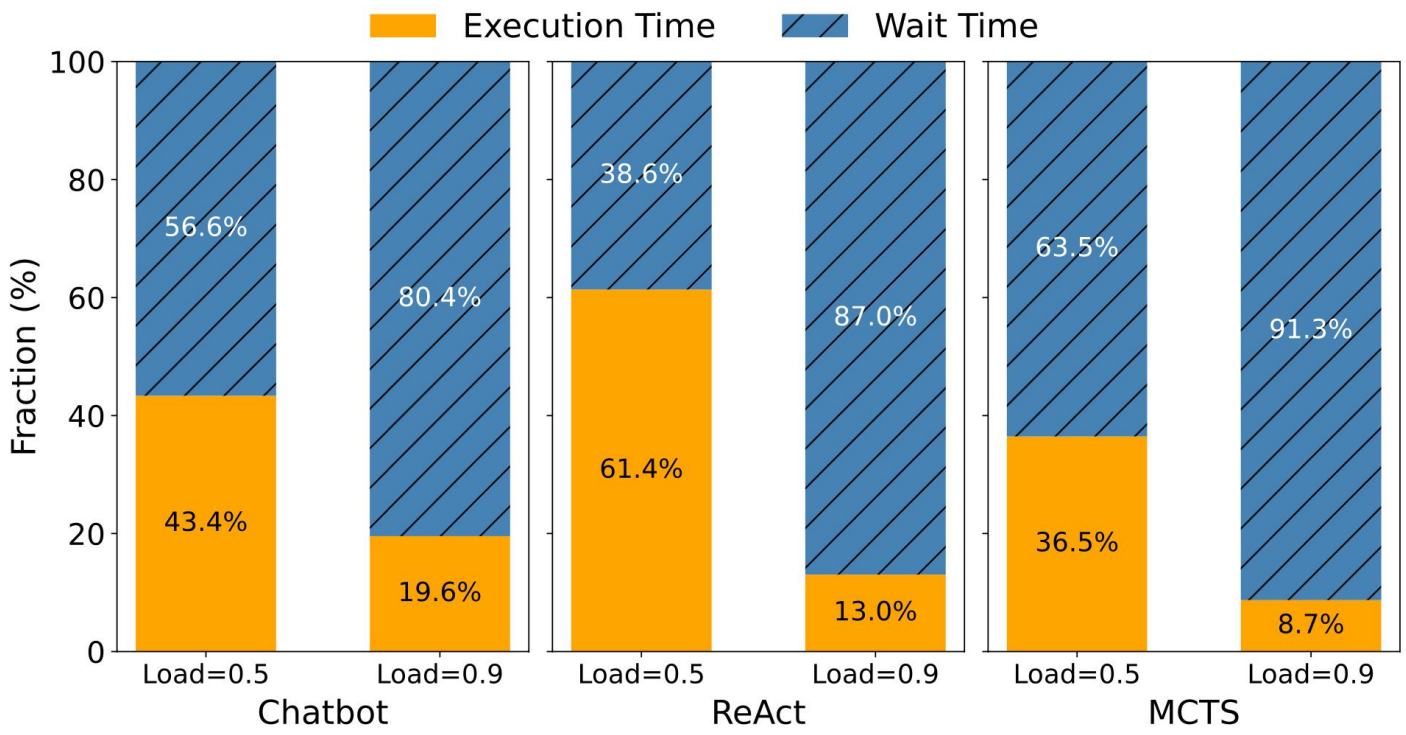
1 研究背景与现状

2 研究机遇与挑战

3 研究内容

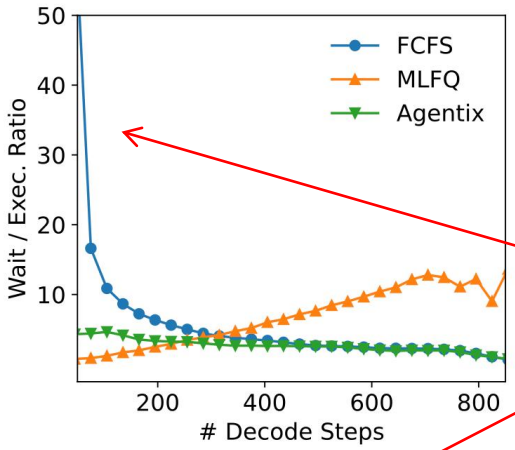
4 实验与总结

机遇1：利用 Program 的信息做推理调度优化

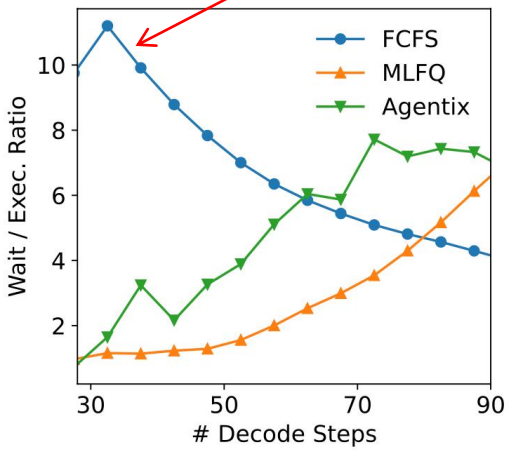


实验发现不同的负载强度下 执行等待时间占比不同，随着负载变强，等待时间成为主要时间

机遇1：利用 Program 的信息做推理调度优化



(a) Chatbot, LLM Calls



(c) MCTS, LLM Calls

调用级别阻塞

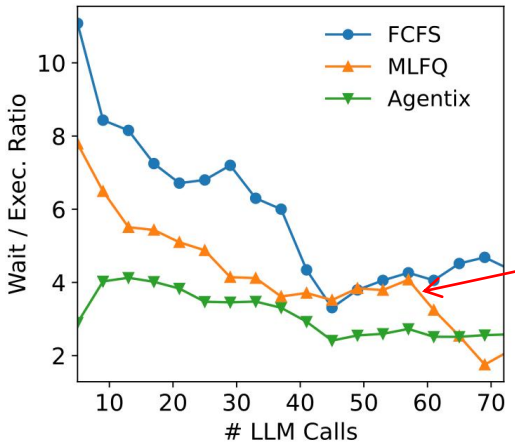
FCFS: 长输出请求占用执行机会，使短调用等待。

- 横轴：LLM 解码的长度
- 纵轴：等待时间 / 执行时间
- FCFS（蓝色线）：先来先服务
- MLFQ（橙色线）：多级反馈队列

传统 OS 的思路：采用抢占的方式减少短任务的饥饿等待

MLFQ：多级反馈队列的方式，当服务时间超过阈值则降级放在下一级的等待队列中

机遇1：利用程序级的信息做推理调度优化

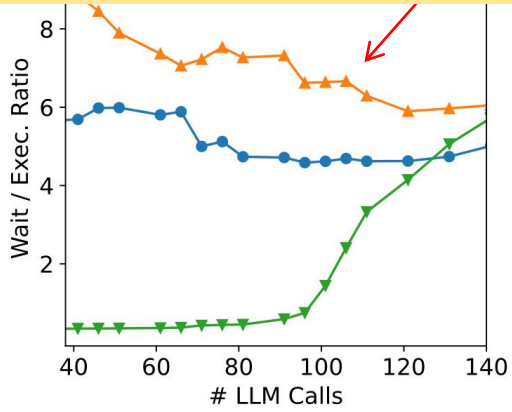


程序级阻塞

多级反馈队列方法性能随调用数增加性能下降严重

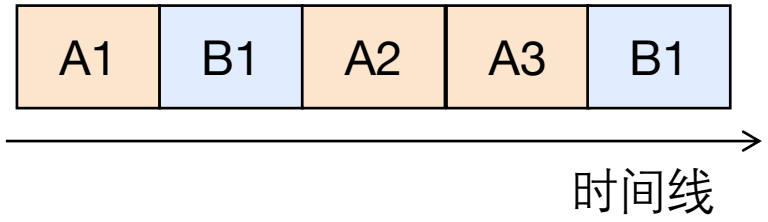
Program	LLM Calls	Decode Steps
A	A1 → A2 → A3	{1, 1, 1}
B	B1	{2}

挑战1：如何在程序结构未知的情况下，进行推理调度优化以提高整体的推理服务性能？



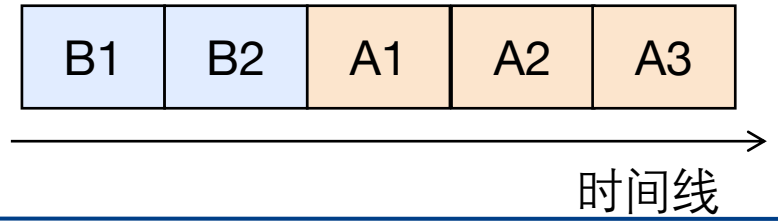
(d) MCTS, Programs

MLFQ



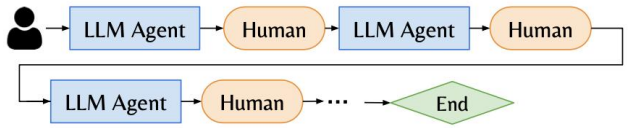
(4 + 5) / 2 = 4.5

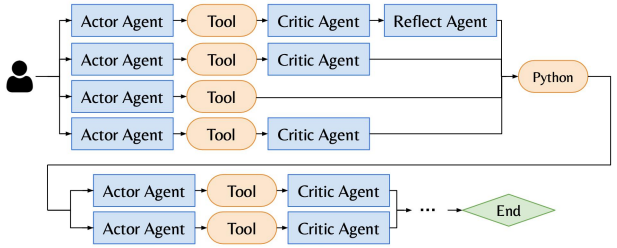
最优方案

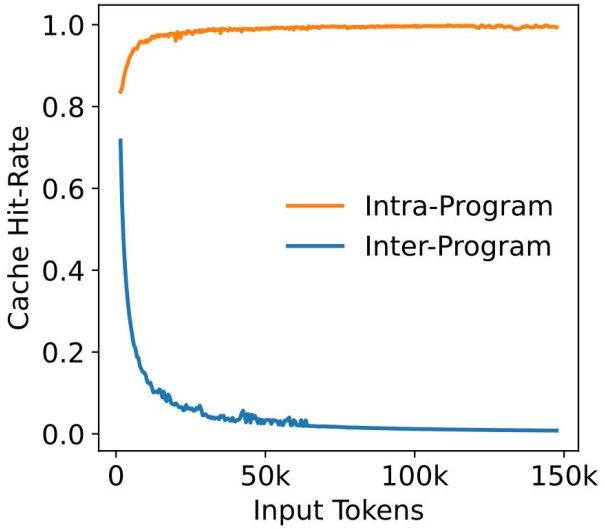


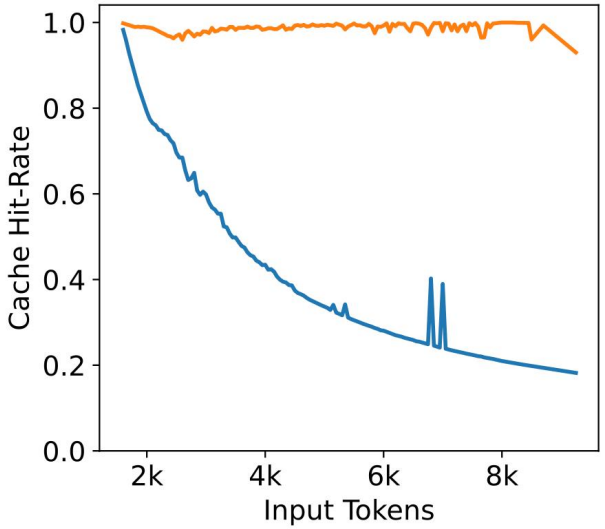
(2 + 5) / 2 = 3.5

机遇2：利用不同程序的前缀匹配特点做负载均衡









(a) Single thread: Chatbot (b) Multiple threads: MCTS

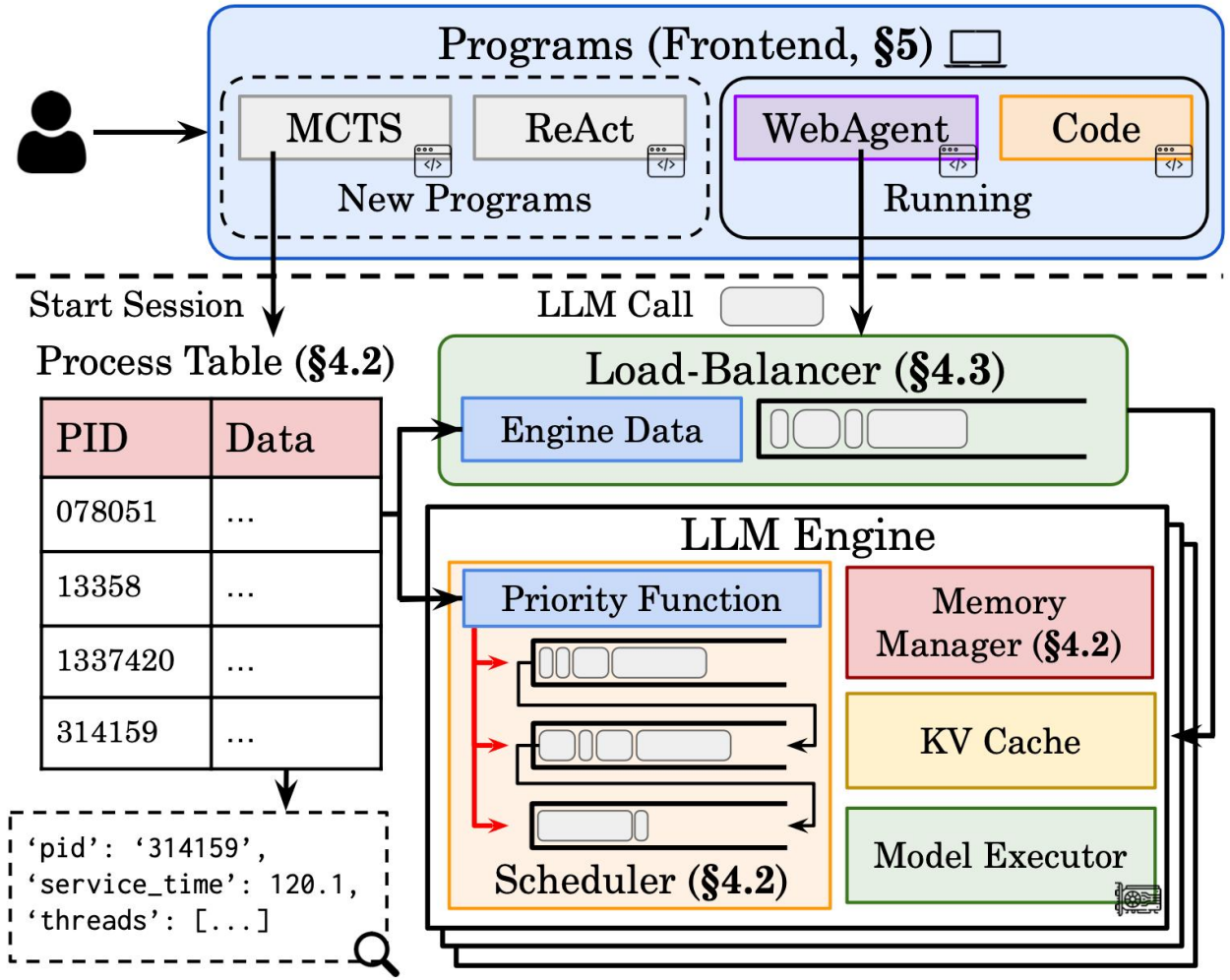
- 同一程序内前缀匹配率会不断累计
- 不同程序间前缀匹配率会随输入 Token 数衰减
- Chatbot 这种形式下程序的调用的前缀匹配率

挑战2：如何利用程序结构特点，在多引擎推理的情况下做负载均衡？

提纲

1 研究背景与现状
2 研究机遇与挑战
3 研究内容
4 实验与总结

Agentix-系统架构



系统假设

- 系统在每次LLM 调用到达时并不知道该程序的完整结构

设计1: 程序级调度优化

设计2: 基于前缀匹配的负载均衡

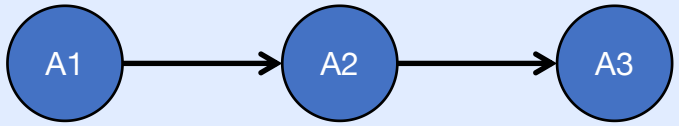
设计1

利用程序**已执行时间**作为优先级信息

为每个程序分配一个独立的优先级标识 P(x)，**P 越大，优先级越低**

单线程

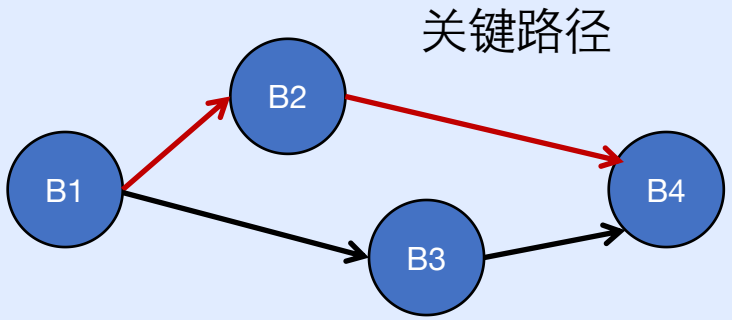
调用结构



推理时间	1	2	1
P(A)	0	0 + 1 = 1	1 + 2 = 3

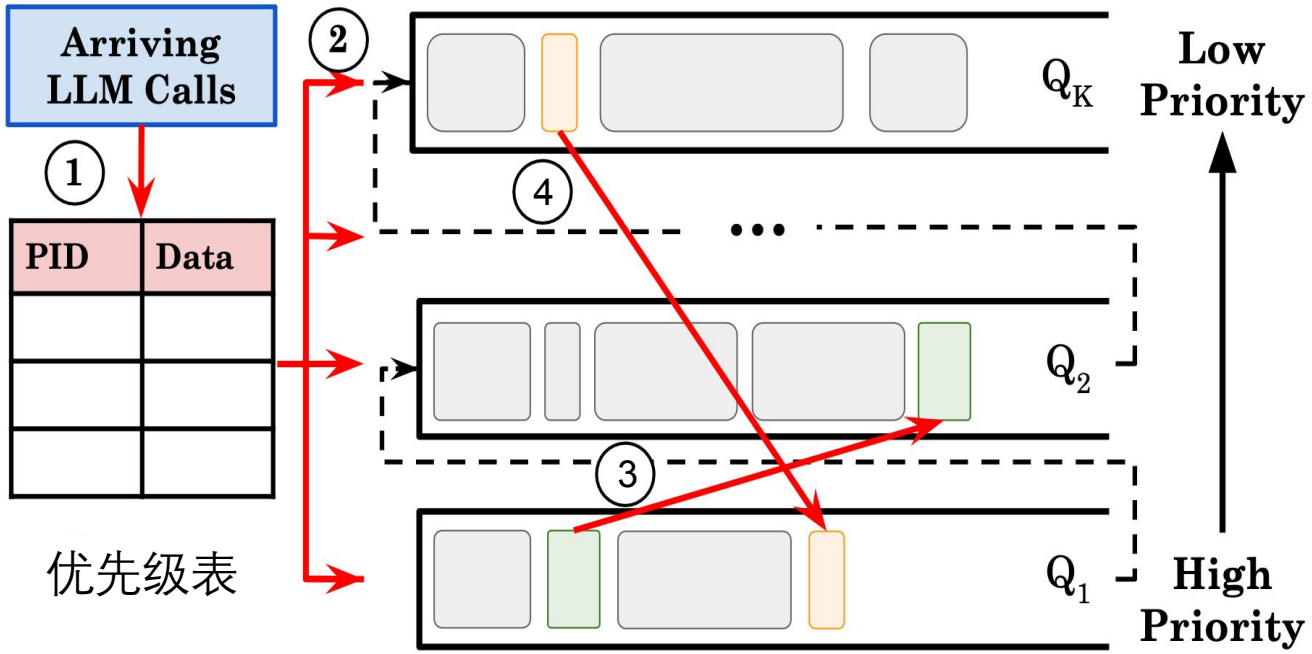
多线程

调用结构



推理时间	1	2	1	1
P(B)	0	0+1=1	0+1=1	2+1=3

设计1



①新请求到达先查看**优先级表**

②按照记录的优先级放在对应的队列中

③当高优先级队列中的请求执行时间过程，则将其放到下一级队列中

④当低优先级任务等待时间过长则调度到高优先级队列中

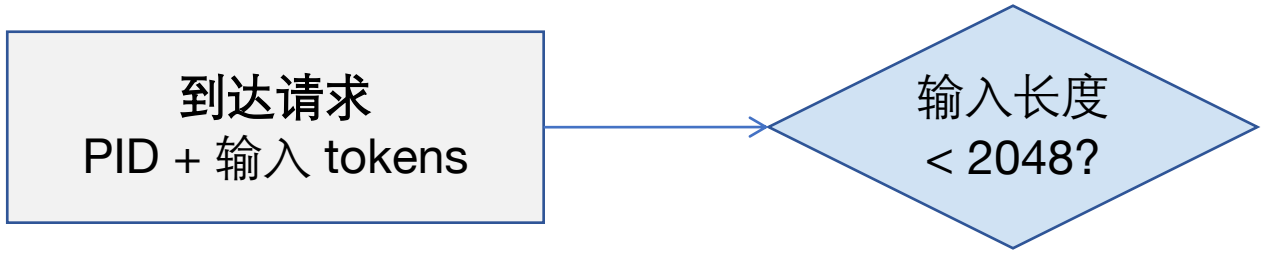
PID: 是每个程序独立的 ID

Data: 优先级信息 P

设计2 程序具备性感知的负载均衡

核心思想： 用程序 ID 考虑缓存局部性，按输入长度选择路由策略

方法：输入长度分流 + 程序绑定



设计2：程序局部性感知的负载均衡

提纲

1 研究背景与现状
2 研究机遇与挑战
3 研究内容
4 实验与总结

实验设置

➢ **实验设备和评估模型**

- 单卡 A100 LLaMA-3.1-8B
- 4卡A100 LLaMA-3.1-70B
- 8卡A100 Falcon-180B

➢ **任务类型**

- Chatbot Agent (单线程): ShareGPT
- ReAct Agent (单线程): BFCL
- MCTS Agent (多线程): LATS
- Mixed： 混合上述三种

➢ **对比方法**

□ **单引擎 (对比 Design1)**

- (SOSP'23) vLLM ： 不开启优化技术
- vLLM-opt ： 开启 Prefix Caching 等优化技术
- MLFQ： vLLM 基础上使用多级反馈队列

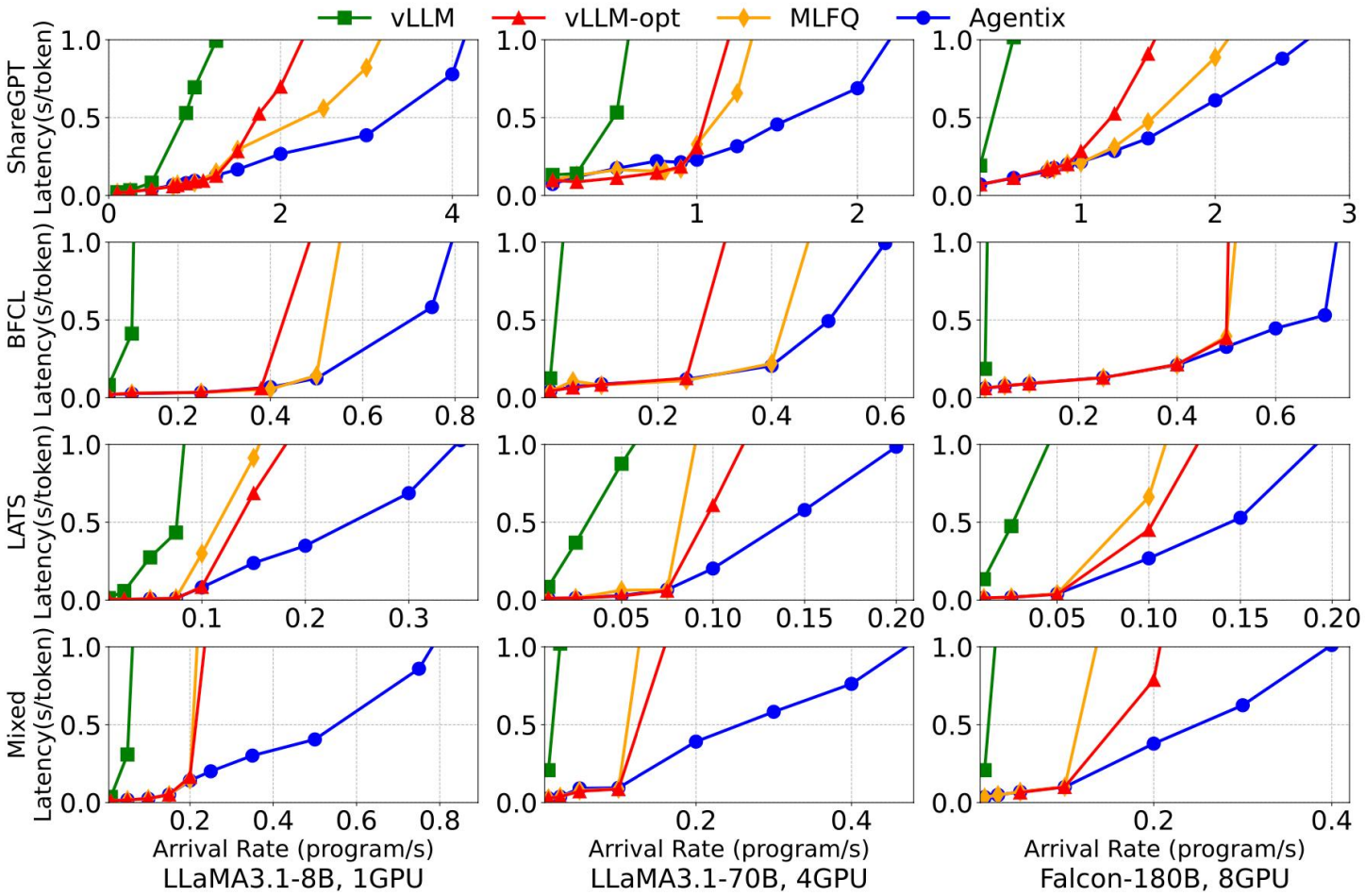
□ **多引擎 (对比 Design2)**

- Round Robin: Naive 轮询设备
- Least Used: Naive 最均衡方法
- (ICLR'25) Preble: SOTA 前缀树负载均衡

➢ **评估指标（越小越好）**

- 程序级别延迟

实验结果：单引擎对比实验



子图间横轴：不同尺寸模型
子图间纵轴：不同任务类型

子图内横轴：程序到达速率，负载强度
子图内纵轴：程序级别延迟

实验表明：

- Agentix 在所有方法上相比 Baseline 有 **1.5x - 15x** 的提升
- 其中在多线程的任务场景下提升最大，对比三种 baseline 分别提升 **15x, 5.5x 和 5x**

图1:四种实际工作负载下的不同LLM服务系统的平均延迟

实验结果：多引擎对比实验

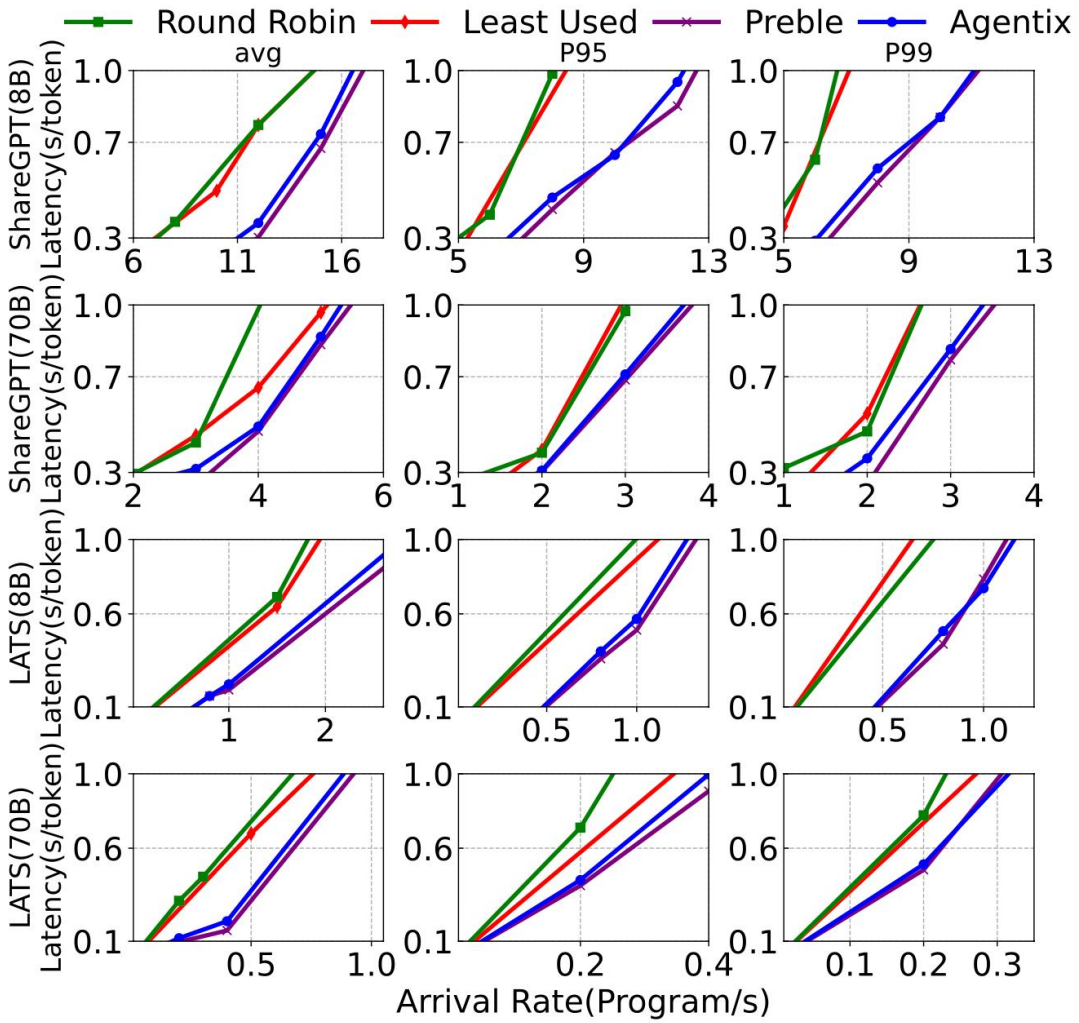


图2:不同工作负载下的不同LLM服务系统的平均延迟

子图间横轴：平均时延P95时延 P99时延
子图间纵轴：不同任务类型和模型

子图内横轴：程序到达速率
子图内纵轴：程序级别延迟

实验表明：

- Agentix 在相比 Naive Baseline 有 **1.4x** 的提升
- 在更低的实现代价下和 SOTA 的复杂前缀树方法的性能相近

实验结果：消融实验

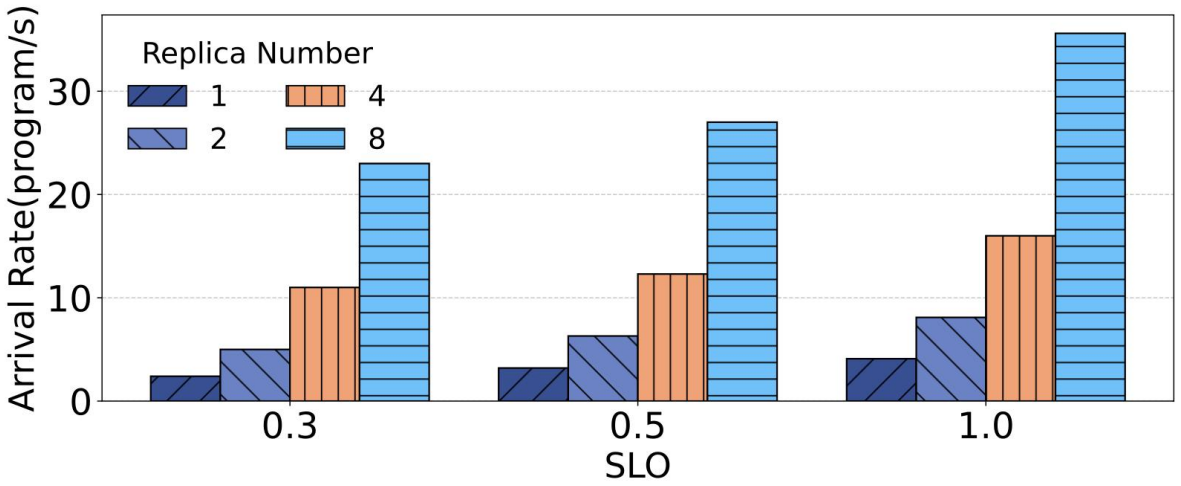


图3:不同 SLO 下能支持的最大负载强度

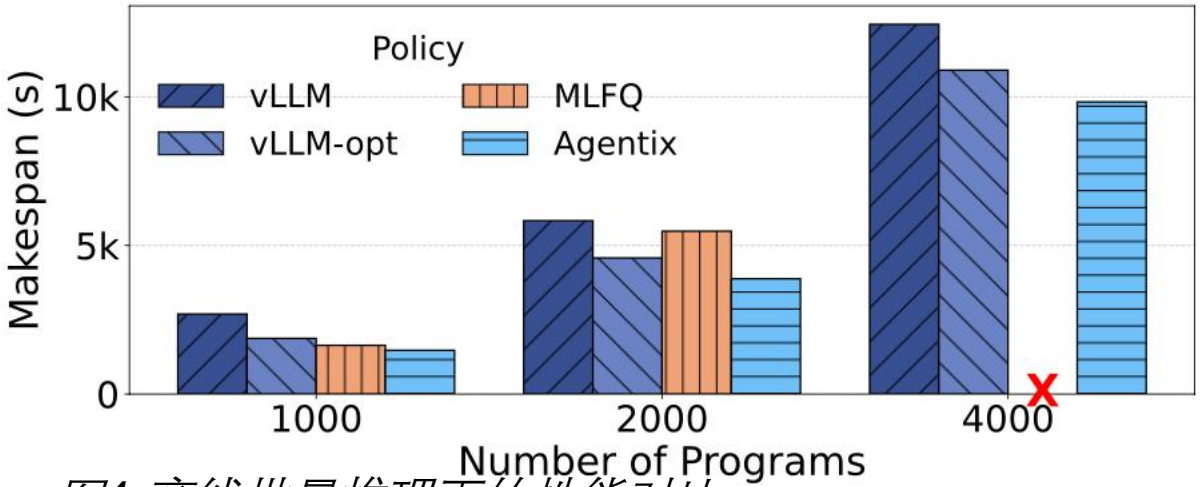


图4:离线批量推理下的性能对比

通过可扩展性实验证明：

- Agentix 通过负载均衡，能保证在设备扩展后带来稳定的性能提升

通过离线实验证明：

- Agentix 始终优于 Baseline，优化总时延达 10-40%

展望

- **这个paper提到的idea，能不能用在自己的方向/project上面？**
  - 我的主要关注在动态AI工作流下调度，可以从模型 + 工具调度的角度考虑将程序级调度的思想引入我们的工作。
  - 虽无法预知完整动态完整结构，但可以利用已展开的调用关系，为后续调度提供先验知识

- **这个paper能不能泛化，需要较为熟悉这个小方向？**
  - 针对端侧内存和能耗限制，可以结合任务不同安排模型与工具执行，根据任务特点保留可复用的上下文，减少推理的开销。
  - 同时考虑在端侧推理任务的服务目标不同而采用不同的优先级形式

- **这个paper有什么问题，基于这个paper还能做什么？**
  - Agentix 主要依据历史服务量分配优先级，可以进一步结合当前依赖和后续调用预测，识别更影响任务完成时间的调用。



東南大學
SOUTHEAST UNIVERSITY

感谢各位老师和同学！
请大家提出宝贵意见！